

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

Tanmay Talanki Rajeeva (1WA23CS011)

in partial fulfillment for the award of the degree of

**BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Dec-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **Tanmay Talanki Rajeeva (1WA23CS011)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Dr. Shwetha K S Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

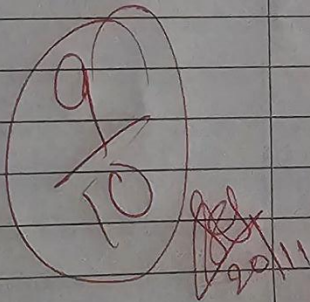
Index

Sl. No.	Date	Experiment Title	Page No.
1	21/8/25	Genetic algorithm	
2	28/8/25	Gene Expression Algorithm	
3	11/9/25	Particle Swarm Optimization	
4	9/10/25	Ant Colony Optimization	
5	16/10/25	Cuckoo Search	
6	6/11/25	Grey Wolf Optimizer	
7	13/11/25	Parallel Cellular Algorithms	

Github Link:

<https://github.com/TRTanmay/BIS-Lab>

School/College Tel. No. _____ Parents Tel. No. _____

Sl. No.	Date	Title		Page No.	Teacher Sign / Remarks
1	24/8	LAB-1	GA	9M	}
2	25/8	LAB-2	GEA	8m	
3	11/9	LAB-3	PSO	8m	}
4	9/10	LAB-4	ALO	8m	
5	16/10	LAB-5	CSA	10M	}
6	6/11	LAB-6	GW0	10M	
7	13/11	LAB-7	PCO	10M	
					

Program 1

Genetic Algorithms (GA) are inspired by the process of natural selection and genetics, where the fittest individuals are selected for reproduction to produce the next generation. GAs are widely used for solving optimization and search problems. Implement a Genetic Algorithm using Python to solve a basic optimization problem, such as finding the maximum value of a mathematical function.

Algorithm:

Genetic algorithm for optimization problems

It is a technique where computers search for better soln. by imitating the process of natural evolution. It begins with a group of random possible soln. which are tested for their performance & the parents are combined using crossover to form new solutions & sometimes small mutations are added to maintain variety. By repeating this process many times. The overall quality of soln. improve step by step.

Applications:

1) Genetic Algorithm

- Eng. design optimization
- Machine learning
- Scheduling
- Automated game strategy dev.

2) Particle Swarm Optimization

- Nonlinear function optimization
- Power systems
- Robotics

3) Ant colony Optimization:

- Network routing
- Supply chain
- Vehicle routing problems
- Scheduling tasks

4) cuckoo Search

- Eng. design problems
- Data mining
- Biomedical engineering

5) Grey Wolf optimizer (GWO)

- Control system design
- Image processing
- Power system optimization
- MLK parameter selection

6) Parallel Cellular Algo

- Large-scale optimization
- Distributed computing
- Image analysis
- Cryptography

7) Gene Expression Algorithm

- Symbolic regression
- Function approx.
- Forecasting

Genetic Algorithm

- 1) Select initial population
- 2) Calculate fitness function
- 3) Selecting mating pool
- 4) Crossover
- 5) Mutation

Crossover:

a) Single-point: c) Uniform-point

P1 = 11100 (23) 50% of each parent

P2 = 10101 (21)

P1 = 11 | 100

P2 = 10 | 101

P1 = 11 | 101

P2 = 10 | 100

P1 = 11101 (29)

P2 = 10100 (20)

b) 2-point cross-over:

P1 = 01010 (10)

P2 = 11001 (25)

P1 = 0 | 101 | 0

P2 = 1 | 100 | 1

P1 = 0 | 100 | 0 P1 = 01000 (8)

P2 = 1 | 101 | 1 P2 = 11011 (27)

Problem Formulas

- Probability = $\frac{f(x)}{\sum f(x)}$

- Expected outcome = $\frac{f(x)}{\text{avg}(\sum f(x))}$

String no.	Init. population	α value	x^*	Probability	Expected outcome
1	01100	12	166	0.1267	12.67
2	11001	25	625	0.341	54.1
3	00101	5	25	0.0216	2.16
4	10011	19	361	0.3126	31.26

Sum = 1155

Avg = 288.75

Maximum = 625

Selecting mating pool

String no.	Mating pool	Crossover point	Offspring after crossover	α value	Fitness function: x^*
1	01100	4	01101	13	169
2	11001	4	11000	24	576
3	11001	2	11011	27	729
4	10011	2	10001	17	289

Sum = 1763

Avg = 440.75

Maximum = 729

Mutation:

String no.	Offspring after crossover	Mutation chromosomes by flipping	Offspring after mutation	α value	x^*
1	01101	10000	11101	29	841
2	11000	00000	11000	24	576
3	11011	00000	11011	27	729
4	10001	00101	10100	20	400

Code

```

import random
CHROM_LENGTH = 5
CROSS_RATE = 0.8
MUT_RATE = 0.1

def fitness(n):
    return n**2

def decode(b):
    return int(b, 2)

def encode(n):
    return format(n, '0' + str(CHROM_LENGTH) + 'b')

def select(pop, fitness_vals):
    total_fitness = sum(fitness_vals)
    pick = random.uniform(0, total_fitness)
    curr = 0
    for i, f in enumerate(fitness_vals):
        curr += f
        if curr > pick:
            return pop[i]
    return pop[-1]

```



```

def crossover(p1, p2):
    if random.random() < CROSS_RATE:
        point = random.randint(1, CHROM_LEN-1)
        c1 = p1[:point] + p2[point:]
        c2 = p2[:point] + p1[point:]
        return c1, c2
    return p1, p2

def mutate(c):
    c_list = list(c)
    for i in range(CHROM_LEN):
        if random.random() < MUT_RATE:
            c_list[i] = '1' if c_list[i] == '0' else '0'
    return ''.join(c_list)

def ga():
    user_input = input("Enter initial population (e.g. 1 2 3): ")
    init_vals = list(map(int, user_input.split()))
    gens = int(input("Enter no. of generations: "))
    pop_size = len(init_vals)
    pop = [encode(n) for n in init_vals]

    print("\n Initial population: ", pop, [decode(c) for c in pop])

    for gen in range(1, gens+1):
        decoded_pop = [decode(c) for c in pop]
        fitness_vals = [Fitness(n) for n in decoded_pop]

        total_fitness = sum(fitness_vals)
        avg_fitness = sum(fitness_vals) / len(fitness_vals)

        print(f"\n Generation {gen}: ")
        for i in range(pop_size):
            print(f"Chromosome {i+1}: {pop[i]}, ")
            decoded = decode(pop[i])
            fitness = Fitness(decoded)
            prob = (fitness / total_fitness) * 32
            print(f"Fitness: {fitness}, Prob: {prob}")

        new_pop = []
        while len(new_pop) < pop_size:
            p1 = select(pop, fitness_vals)
            p2 = select(pop, fitness_vals)
            c1, c2 = crossover(p1, p2)
            new_pop.extend([mutate(c1), mutate(c2)])
        pop = new_pop[:pop_size]

        decoded_pop = [decode(c) for c in pop]
        fitness_vals = [Fitness(n) for n in decoded_pop]
        best_idx = fitness_vals.index(max(fitness_vals))
        print("\n Final solution: ")
        print(f"Chromosome: ", pop[best_idx],
              "Decoded: ", decoded_pop[best_idx],
              "Fitness: ", fitness_vals[best_idx])

ga()

```

Code:

```
import random
```

```
population_size = 4
```

```
chromosome_length = 5
```

```
initial_population = []
```

```
print(f"Enter the {population_size} initial chromosomes (each of length {chromosome_length}):")
```

```
for i in range(population_size):
```

```
    while True:
```

```
        chromosome = input(f"Enter chromosome {i+1}: ")
```

```
        if len(chromosome) == chromosome_length and all(c in '01' for c in chromosome):
```

```
            initial_population.append(chromosome)
```

```
            break
```

```
        else:
```

```
            print(f"Invalid input. Please enter a binary string of length {chromosome_length}.")
```

```
print("\nInitial Population:")
```

```
for i, chromosome in enumerate(initial_population):
```

```
    print(f"String no. {i+1}: {chromosome}")
```

```
fitness_values = []
```

```

x_values = []

for chromosome in initial_population:
    x = int(chromosome, 2)
    x_values.append(x)

    fitness = x**2
    fitness_values.append(fitness)

print("\nFitness Calculation:")
print("String no.    Initial population    x value    Fitness")
print("function = x^2")
for i in range(population_size):

    print(f'{i+1:<16} {initial_population[i]:<24} {x_values[i]:<16} {fitness_values[i]}')

sum_fitness = sum(fitness_values)
average_fitness = sum_fitness / population_size
maximum_fitness = max(fitness_values)

print(f'\nSum = {sum_fitness}')
print(f'Average = {average_fitness}')
print(f'Maximum = {maximum_fitness}')

print("\nProbability and Expected Outcome:")
print("String no.    Probability    Prob %    Expected outcome")
for i in range(population_size):
    probability = fitness_values[i] / sum_fitness
    prob_percent = probability * 100
    expected_outcome = fitness_values[i] / average_fitness

    print(f'{i+1:<16} {probability:<24.4f} {prob_percent:<16.2f} {expected_outcome:<16.4f}')

```


Output:

Enter the 4 initial chromosomes (each of length 5):

Enter chromosome 1: 01100

Enter chromosome 2: 11001

Enter chromosome 3: 00101

Enter chromosome 4: 10011

Initial Population:

String no. 1: 01100

String no. 2: 11001

String no. 3: 00101

String no. 4: 10011

Fitness Calculation:

String no.	Initial population	x value	Fitness function = x^2
1	01100	12	144
2	11001	25	625
3	00101	5	25
4	10011	19	361

Sum = 1155

Average = 288.75

Maximum = 625

Probability and Expected Outcome:

String no.	Probability	Prob %	Expected outcome
1	0.1247	12.47	0.4987
2	0.5411	54.11	2.1645
3	0.0216	2.16	0.0866
4	0.3126	31.26	1.2502

Program 2

Gene Expression Algorithms (GEA) are inspired by the biological process of gene expression in living organisms. This process involves the translation of genetic information encoded in DNA into functional proteins. In GEA, solutions to optimization problems are encoded in a manner similar to genetic sequences. The algorithm evolves these solutions through selection, crossover, mutation, and gene expression to find optimal or near-optimal solutions. GEA is effective for solving complex optimization problems in various domains, including engineering, data analysis, and machine learning.

Algorithm:

```
2) Gene Expression Algorithm

Code:
import random
CHROM_LENGTH = 15
CODON_LENGTH = 3
POP_SIZE = 4
CROSS_RATE = 0.8
MUT_RATE = 0.1
GENS = 5

def fitness(phenotype):
    return sum(x**2 for x in phenotype)

def encode(codons):
    return ''.join(format(c, '0' + str(CODON_LENGTH) + 'b') for c in codons)

def decode(chrom):
    return [int('chrom[i:i + CODON_LENGTH]:2') for i in range(0, CHROM_LENGTH, CODON_LENGTH)]

def roulette_selection(pop, fits):
    pick = random.uniform(0, sum(fits))
    current = 0
    for c, f in zip(pop, fits):
        current += f
        if current > pick:
            return c
    return pop[-1]

def crossover(p1, p2):
    if random.random() < CROSS_RATE:
        point = random.randint(1, CHROM_LENGTH - 1)
        return p1[:point] + p2[point:], p2[:point] + p1[point:]
    return p1, p2

def mutate(chrom):
    return ''.join('1' if (bit == '0' and random.random() < MUT_RATE) else bit for bit in chrom)

def GEA():
    population = [encode([random.randint(0, 7) for _ in range(CHROM_LENGTH // CODON_LENGTH)]) for _ in range(POP_SIZE)]
    for _ in range(GENS):
        phenotypes = [decode(c) for c in population]
        fitnesses = [fitness(p) for p in phenotypes]
        new_pop = []
        while len(new_pop) < POP_SIZE:
            p1 = roulette_selection(population, fitnesses)
            p2 = roulette_selection(population, fitnesses)
            c1, c2 = crossover(p1, p2)
            new_pop += [mutate(c1), mutate(c2)]
        population = new_pop[:POP_SIZE]
```

```
phenotypes = [decode(c) for c in population]
fitnesses = [fitness(p) for p in phenotypes]
best = fitnesses.index(max(fitnesses))
print("Best phenotype:", phenotypes[best],
      "Fitness:", fitnesses[best])

GEA()

Output:
Best phenotype: [7, 6, 6, 7, 5] Fitness: 195
```

Code:

```
import random
import math

CHROM_LENGTH = 5
POP_SIZE = 4
CROSS_RATE = 0.8
MUT_RATE = 0.1
GENERATIONS = 10

def fitness(x):
    return -(10 + (x**2 - 10 * math.cos(2 * math.pi * x)))

def encode(x):
    return format(x, f'0{CHROM_LENGTH}b')

def decode(b):
    return int(b, 2)

def gene_expression(chrom):
    return decode(chrom)

def roulette_selection(pop, fitnesses):
    total_fit = sum(fitnesses)
    pick = random.uniform(0, total_fit)
    current = 0
    for i, f in enumerate(fitnesses):
        current += f
        if current > pick:
            return pop[i]
    return pop[-1]

def crossover(p1, p2):
    if random.random() < CROSS_RATE:
        point = random.randint(1, CHROM_LENGTH - 1)
        c1 = p1[:point] + p2[point:]
        c2 = p2[:point] + p1[point:]
        return c1, c2
    return p1, p2

def mutate(chrom):
    chrom_list = list(chrom)
    for i in range(CHROM_LENGTH):
        if random.random() < MUT_RATE:
            chrom_list[i] = '1' if chrom_list[i] == '0' else '0'
    return ''.join(chrom_list)

def gene_expression_algorithm():
    population = [encode(x) for x in [7, 15, 22, 28]]
    print("Initial Population:", population, [decode(c) for c in
population])
```

```

for gen in range(1, GENERATIONS + 1):
    expressed = [gene_expression(c) for c in population]
    fitnesses = [fitness(x) for x in expressed]
    total_fit = sum(fitnesses)
    probs = [f / total_fit for f in fitnesses]
    expected = [p * POP_SIZE for p in probs]
    print(f"\nGeneration {gen}")

    for i in range(POP_SIZE):
        print(f"x={expressed[i]}, bin={population[i]},
fit={fitnesses[i]:.3f}, "
        f"prob={probs[i]:.3f}, exp_count={expected[i]:.2f}")

    new_pop = []
    while len(new_pop) < POP_SIZE:
        p1 = roulette_selection(population, fitnesses)
        p2 = roulette_selection(population, fitnesses)
        c1, c2 = crossover(p1, p2)
        c1, c2 = mutate(c1), mutate(c2)
        new_pop.extend([c1, c2])
    population = new_pop[:POP_SIZE]
    expressed = [gene_expression(c) for c in population]
    fitnesses = [fitness(x) for x in expressed]
    best_idx = fitnesses.index(max(fitnesses))
    print("\nfinal best sol:", expressed[best_idx],
population[best_idx], "fitness=",
        fitnesses[best_idx])

gene_expression_algorithm()

```

Output:

```

Initial Population: ['00111', '01111', '10110', '11100'] [7, 15, 22, 28]

Generation 1
x=7, bin=00111, fit=-49.000, prob=0.032, exp_count=0.13
x=15, bin=01111, fit=-225.000, prob=0.146, exp_count=0.58
x=22, bin=10110, fit=-484.000, prob=0.314, exp_count=1.26
x=28, bin=11100, fit=-784.000, prob=0.508, exp_count=2.03

Final Best Solution: 1 00001 fitness= -1.0

Generation 2
x=7, bin=00111, fit=-49.000, prob=0.080, exp_count=0.32
x=6, bin=00110, fit=-36.000, prob=0.059, exp_count=0.23
x=23, bin=10111, fit=-529.000, prob=0.860, exp_count=3.44
x=1, bin=00001, fit=-1.000, prob=0.002, exp_count=0.01

Final Best Solution: 3 00011 fitness= -9.0

Generation 3
x=12, bin=01100, fit=-144.000, prob=0.574, exp_count=2.29
x=3, bin=00011, fit=-9.000, prob=0.036, exp_count=0.14
x=7, bin=00111, fit=-49.000, prob=0.195, exp_count=0.78
x=7, bin=00111, fit=-49.000, prob=0.195, exp_count=0.78

Final Best Solution: 3 00011 fitness= -9.0

Generation 4
x=7, bin=00111, fit=-49.000, prob=0.040, exp_count=0.16
x=3, bin=00011, fit=-9.000, prob=0.007, exp_count=0.03
x=14, bin=01110, fit=-196.000, prob=0.161, exp_count=0.65
x=31, bin=11111, fit=-961.000, prob=0.791, exp_count=3.16

Final Best Solution: 5 00101 fitness= -25.0

```


Generation 5
x=7, bin=00111, fit=-49.000, prob=0.059, exp_count=0.24
x=23, bin=10111, fit=-529.000, prob=0.639, exp_count=2.56
x=15, bin=01111, fit=-225.000, prob=0.272, exp_count=1.09
x=5, bin=00101, fit=-25.000, prob=0.030, exp_count=0.12

Final Best Solution: 3 00011 fitness= -9.0

Generation 6
x=5, bin=00101, fit=-25.000, prob=0.189, exp_count=0.76
x=7, bin=00111, fit=-49.000, prob=0.371, exp_count=1.48
x=3, bin=00011, fit=-9.000, prob=0.068, exp_count=0.27
x=7, bin=00111, fit=-49.000, prob=0.371, exp_count=1.48

Final Best Solution: 4 00100 fitness= -16.0

Generation 7
x=5, bin=00101, fit=-25.000, prob=0.086, exp_count=0.34
x=4, bin=00100, fit=-16.000, prob=0.055, exp_count=0.22
x=15, bin=01111, fit=-225.000, prob=0.773, exp_count=3.09
x=5, bin=00101, fit=-25.000, prob=0.086, exp_count=0.34

Final Best Solution: 5 00101 fitness= -25.0

Generation 8
x=5, bin=00101, fit=-25.000, prob=0.099, exp_count=0.40
x=5, bin=00101, fit=-25.000, prob=0.099, exp_count=0.40
x=9, bin=01001, fit=-81.000, prob=0.321, exp_count=1.29
x=11, bin=01011, fit=-121.000, prob=0.480, exp_count=1.92

Final Best Solution: 1 00001 fitness= -1.0

Generation 9
x=1, bin=00001, fit=-1.000, prob=0.002, exp_count=0.01
x=21, bin=10101, fit=-441.000, prob=0.942, exp_count=3.77
x=1, bin=00001, fit=-1.000, prob=0.002, exp_count=0.01
x=5, bin=00101, fit=-25.000, prob=0.053, exp_count=0.21

Final Best Solution: 1 00001 fitness= -1.0

Generation 10
x=1, bin=00001, fit=-1.000, prob=0.250, exp_count=1.00
x=1, bin=00001, fit=-1.000, prob=0.250, exp_count=1.00
x=1, bin=00001, fit=-1.000, prob=0.250, exp_count=1.00
x=1, bin=00001, fit=-1.000, prob=0.250, exp_count=1.00

Final Best Solution: 1 00001 fitness= -1.0

Program 3

Particle Swarm Optimization (PSO) is inspired by the social behavior of birds flocking or fish schooling. PSO is used to find optimal solutions by iteratively improving a candidate solution with regard to a given measure of quality. Implement the PSO algorithm using Python to optimize a mathematical function.

Algorithm:

Date 11/9/25
Page _____

Particle Swarm Optimization (PSO)

Application:
Neural Network training.

- PSO can be used to train neural networks by treating networks weights & biases
- PSO searches for optimal weight configuration that minimize a loss function.
- Each particle in swarm represents a candidate set of weights & fitness is network's loss on training data

$$v_i^{t+1} = v_i^t + c_1 U_i^t (p_{best,i}^t - p_i^t) + c_2 U_i^t (g_{best}^t - p_i^t)$$

Velocity of particles:

$$v_i^{t+1} = v_i^t + c_1 r_1^t [p_{best,i}^t - x_i^t] + c_2 r_2^t [G_{best}^t - x_i^t]$$

where $i = 1, \dots, 9$

Fitness function:

$$f(x, y) = x^2 + y^2$$

```

Code:
import random
def objective(pos):
    return pos[0]**2 + pos[1]**2
num_particles = 5
num_iterations = 10
w = 0.5
c1 = 1.0
c2 = 1.0
positions = [[random.uniform(-5, 5), random.uniform(-5, 5)]
              for _ in range(num_particles)]
velocities = [[random.uniform(-1, 1), random.uniform(-1, 1)]
              for _ in range(num_particles)]

pbests = [pos[:] for pos in positions]
pbest_scores = [objective(p) for p in positions]
gbest_score = max(pbest_scores)
gbest = pbests[pbest_scores.index(gbest_score)][:]
for it in range(num_iterations):
    for i in range(num_particles):
        for d in range(2):
            r1 = random.random()
            r2 = random.random()
            velocities[i][d] = (
                w * velocities[i][d] +
                c1 * r1 * (pbests[i][d] - positions[i][d]) +
                c2 * r2 * (gbest[d] - positions[i][d])
            )
            positions[i][d] += velocities[i][d]
        score = objective(positions[i])
        if score < pbest_scores[i]:
            pbests[i] = positions[i][:]
            pbest_scores[i] = score

```

```

gbest_score = max(pbest_scores)
gbest = pbests[pbest_scores.index(gbest_score)][:]
print(f"Iteration {it+1}: Best fitness value: {gbest_score}")

print("\n Final drone waypoints:")
for i, pos in enumerate(positions):
    print(f"Drone {i+1}: {round(coord, 2) for coord in pos}")

print(f"Final global best score (minimum objective value):",
      round(gbest_score, 2))

```

Output:

Output:

Code:

```

import random
def objective(pos):
    return pos[0]**2 + pos[1]**2

```

```

num_particles = 5
num_iterations = 10
w = 0.5
c1 = 1.0
c2 = 1.0
positions = [[random.uniform(-5, 5), random.uniform(-5, 5)] for _ in range(num_particles)]
velocities = [[random.uniform(-1, 1), random.uniform(-1, 1)] for _ in range(num_particles)]

```

```

pbests = [pos[:] for pos in positions]
pbest_scores = [objective(p) for p in positions]
gbest_score = max(pbest_scores)
gbest = pbests[pbest_scores.index(gbest_score)][:]
for it in range(num_iterations):
    for i in range(num_particles):
        for d in range(2):
            r1 = random.random()
            r2 = random.random()

```

```

        velocities[i][d] = (
            w * velocities[i][d]
            + c1 * r1 * (pbests[i][d] - positions[i][d])
            + c2 * r2 * (gbest[d] - positions[i][d])
        )
        positions[i][d] += velocities[i][d]

    score = objective(positions[i])

    if score > pbest_scores[i]:
        pbests[i] = positions[i][:]
        pbest_scores[i] = score

    gbest_score = max(pbest_scores)
    gbest = pbests[pbest_scores.index(gbest_score)][:]
    print(f'Iteration {it+1}: Best fitness value = {gbest_score}')

print("\nFinal drone waypoints:")
for i, pos in enumerate(positions):
    print(f'Drone {i+1}: {[round(coord, 2) for coord in pos]}')
print("Final global best score (maximum objective value):", round(gbest_score, 2))

```

Output:

```

Iteration 1: Best fitness value = 42.55830242682573
Iteration 2: Best fitness value = 48.303298134522166
Iteration 3: Best fitness value = 70.66118207617731
Iteration 4: Best fitness value = 94.70344213061361
Iteration 5: Best fitness value = 127.98447565143512
Iteration 6: Best fitness value = 151.81784430205846
Iteration 7: Best fitness value = 175.85801363745017
Iteration 8: Best fitness value = 199.88130930513927
Iteration 9: Best fitness value = 212.47068627292361
Iteration 10: Best fitness value = 225.79235152703808

```

Final drone waypoints:

Drone 1: [-12.63, 7.76]

Drone 2: [-12.55, 7.83]

Drone 3: [-12.74, 7.44]

Drone 4: [-3.06, 1.65]

Drone 5: [-12.89, 7.72]

Final global best score (maximum objective value): 225.79

Program 4

The foraging behavior of ants has inspired the development of optimization algorithms that can solve complex problems such as the Traveling Salesman Problem (TSP). Ant Colony Optimization (ACO) simulates the way ants find the shortest path between food sources and their nest. Implement the ACO algorithm using Python to solve the TSP, where the objective is to find the shortest possible route that visits a list of cities and returns to the origin city.

Algorithm:

Date 9/12/25
Page _____

Ant Colony Optimization

Application:
Sequenced Ordering Problem (SOP)

- It is an extension of TSP where tasks / places have a specific order they must follow.
- In addition:

Date / /
Page _____

Code:

```
import random

def heuristic(i, j):
    return random.uniform(1, 10)

def objective(solution, dist_matrix, precedence_constraints):
    total_cost = 0
    for i in range(len(solution)-1):
        total_cost += dist_matrix[solution[i]][solution[i+1]]
    penalty = 0
    for i in range(1, len(solution)):
        for j in range(i):
            if precedence_constraints[solution[i]] <
               precedence_constraints[solution[j]]:
                penalty += 100
    total_cost += penalty
    return total_cost

num_ants = 10
num_iterations = 10
num_tasks = 5
alpha = 1.0
beta = 2.0
rho = 0.1
tau_0 = 0.1
pheromone_matrix = [[tau_0 for _ in range(num_tasks)]
                    for _ in range(num_tasks)]
```

```

def aco_algo():
    global pheromone_matrix
    best_solution = None
    best_cost = float('inf')
    iteration_best_costs = []
    for iteration in range(num_iterations):
        all_solutions = []
        all_costs = []
        for ant in range(num_ants):
            solution = []
            unvisited = list(range(num_tasks))
            current_task = random.choice(unvisited)
            solution.append(current_task)
            unvisited.remove(current_task)
            while unvisited:
                probabilities = []
                for task in unvisited:
                    pheromone = pheromone_matrix[current_task][task]**alpha
                    heuristic_value = heuristic(current_task, task)**beta
                    probability = pheromone * heuristic_value
                    probabilities.append(probability)
                total_prob = sum(probabilities)
                probabilities = [p/total_prob for p in probabilities]
                next_task = random.choices(unvisited, probabilities)[0]
                solution.append(next_task)
                unvisited.remove(next_task)
            current_task = next_task
            solution_cost = objective(solution, distance_matrix, precedence_constraints)
            all_solutions.append(solution)
            all_costs.append(solution_cost)
            if solution_cost < best_cost:
                best_solution = solution
                best_cost = solution_cost

        for i in range(num_tasks):
            for j in range(num_tasks):
                pheromone_matrix[i][j] *= (1 - rho)
        for ant in range(num_ants):
            for i in range(len(all_solutions[ant])-1):
                task1 = all_solutions[ant][i]
                task2 = all_solutions[ant][i+1]
                pheromone_matrix[task1][task2] += 1/all_costs[ant]
        iteration_best_costs.append(best_cost)
        iteration_best_costs.sort()
        print("Best cost:", best_cost)
        for idx, cost in enumerate(iteration_best_costs, 1):
            print(f"Iteration {idx}: Best cost = {cost:.4f}")
        return best_solution, best_cost

best_solution, best_cost = aco_algo()
print("In Best solution found:", best_solution)
print("Best cost (objective value):", best_cost)

```

Output:

Best costs:

Iter1: Best cost = 122.4889

Iter2: Best cost = 122.4889

Iter3: Best cost = 218.7723

Iter4: Best cost = 218.7723

Iter5: Best cost = 218.7723

Iter6: Best cost = 218.7723

Iter7: Best cost = 221.7103

Iter8: Best cost = 221.7103

Iter9: Best cost = 222.6388

Iter10: Best cost = 222.6388

Code:

```
import random
```

```
def heuristic(i, j):
    return random.uniform(1, 10)
```

```
def objective(solution, distance_matrix, precedence_constraints):
    total_cost = 0
    for i in range(len(solution) - 1):
        total_cost += distance_matrix[solution[i]][solution[i + 1]]
```

```
    penalty = 0
    for i in range(1, len(solution)):
        for j in range(i):
            if precedence_constraints[solution[i]] < precedence_constraints[solution[j]]:
                penalty += 100
```

```
    total_cost += penalty
    return total_cost
```

```
num_ants = 10
num_iterations = 10
```

```

num_tasks = 5
alpha = 1.0
beta = 2.0
rho = 0.1
tau_0 = 0.1

pheromone_matrix = [[tau_0 for _ in range(num_tasks)] for _ in range(num_tasks)]

precedence_constraints = [0, 1, 2, 3, 4]

distance_matrix = [[random.uniform(1, 10) for _ in range(num_tasks)] for _ in range(num_tasks)]

def aco_algorithm():
    global pheromone_matrix
    best_solution = None
    best_cost = float('inf')

    iteration_best_costs = []

    for iteration in range(num_iterations):
        all_solutions = []
        all_costs = []

        for ant in range(num_ants):
            solution = []
            unvisited = list(range(num_tasks))
            current_task = random.choice(unvisited)
            solution.append(current_task)
            unvisited.remove(current_task)

            while unvisited:
                probabilities = []
                for task in unvisited:
                    pheromone = pheromone_matrix[current_task][task] ** alpha
                    heuristic_value = heuristic(current_task, task) ** beta
                    probability = pheromone * heuristic_value
                    probabilities.append(probability)

                total_prob = sum(probabilities)
                probabilities = [p / total_prob for p in probabilities]

                next_task = random.choices(unvisited, probabilities)[0]
                solution.append(next_task)
                unvisited.remove(next_task)
                current_task = next_task

            solution_cost = objective(solution, distance_matrix, precedence_constraints)
            all_solutions.append(solution)
            all_costs.append(solution_cost)

            if solution_cost < best_cost:
                best_solution = solution
                best_cost = solution_cost

    for i in range(num_tasks):
        for j in range(num_tasks):

```

```

        pheromone_matrix[i][j] *= (1 - rho)

    for ant in range(num_ants):
        for i in range(len(all_solutions[ant]) - 1):
            task1 = all_solutions[ant][i]
            task2 = all_solutions[ant][i + 1]
            pheromone_matrix[task1][task2] += 1 / all_costs[ant]

    iteration_best_costs.append(best_cost)

iteration_best_costs.sort()

print("Best Costs :")
for idx, cost in enumerate(iteration_best_costs, 1):
    print(f"Iteration {idx}: Best Cost = {cost:.4f}")

return best_solution, best_cost

best_solution, best_cost = aco_algorithm()

print("\nBest solution found:", best_solution)
print("Best cost (objective value):", best_cost)

```

Output:

```

Best Costs :
Iteration 1: Best Cost = 16.2591
Iteration 2: Best Cost = 16.2591
Iteration 3: Best Cost = 118.6474
Iteration 4: Best Cost = 118.6474
Iteration 5: Best Cost = 118.6474
Iteration 6: Best Cost = 118.6474
Iteration 7: Best Cost = 118.6474
Iteration 8: Best Cost = 118.6474
Iteration 9: Best Cost = 123.3027
Iteration 10: Best Cost = 123.3027

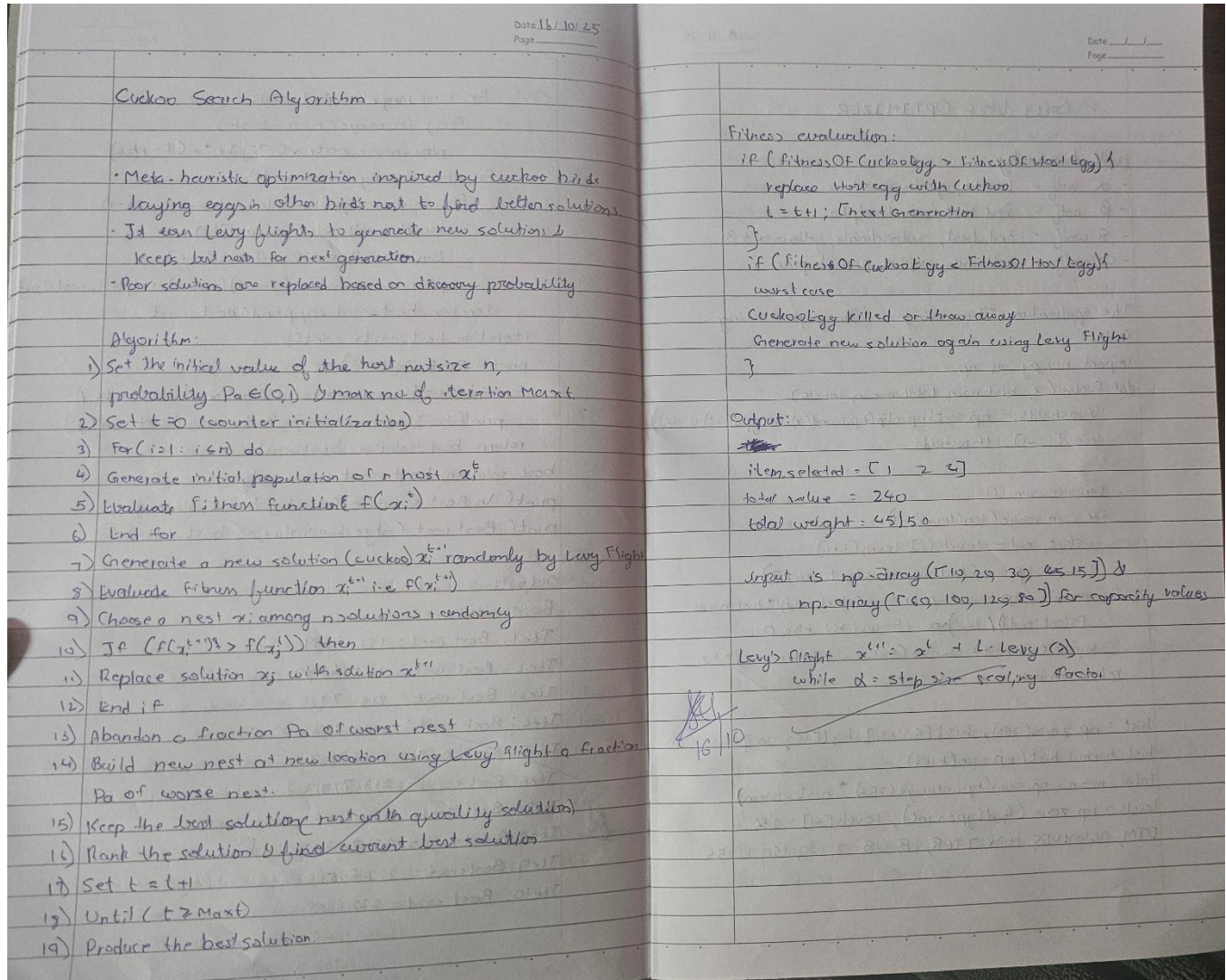
Best solution found: [0, 1, 2, 3, 4]
Best cost (objective value): 16.25912473990214

```


Program 5

Cuckoo Search (CS) is a nature-inspired optimization algorithm based on the brood parasitism of some cuckoo species. This behavior involves laying eggs in the nests of other birds, leading to the optimization of survival strategies. CS uses Lévy flights to generate new solutions, promoting global search capabilities and avoiding local minima. The algorithm is widely used for solving continuous optimization problems and has applications in various domains, including engineering design, machine learning, and data mining.

Algorithm:



The image shows two pages of handwritten notes on the Cuckoo Search Algorithm. The left page, dated 16/10/25, lists key features of the algorithm: it is a meta-heuristic inspired by cuckoo birds laying eggs in other birds' nests to find better solutions; it uses Lévy flights to generate new solutions; and it keeps the best nests for the next generation. It then provides a 19-step algorithm: 1) Set initial values for host nest size n , probability $pa \in (0,1)$, and max iterations $Maxt$; 2) Set $t=0$; 3) For $i=1$ to n , do; 4) Generate initial population of n host x_i^t ; 5) Evaluate fitness function $f(x_i^t)$; 6) End for; 7) Generate a new solution (cuckoo) x_i^{t+1} randomly by Lévy Flight; 8) Evaluate fitness function x_i^{t+1} i.e. $f(x_i^{t+1})$; 9) Choose a nest x_j among n solutions randomly; 10) If $f(x_i^{t+1}) < f(x_j^t)$ then; 11) Replace solution x_j with solution x_i^{t+1} ; 12) End if; 13) Abandon a fraction pa of worst nest; 14) Build new nest at new location using Lévy flight a fraction pa of worse nest; 15) Keep the best solution (nest with quality solution); 16) Rank the solution & find current best solution; 17) Set $t=t+1$; 18) Until $t \geq Maxt$; 19) Produce the best solution. The right page, dated 1/1/25, details the fitness evaluation: if $f(\text{FitnessOfCuckooEgg}) < f(\text{FitnessOfWorstEgg})$, replace the worst egg with the cuckoo, increment t , and if the cuckoo's fitness is less than the host's, the cuckoo is killed and a new solution is generated using Lévy flight. It also lists output values: items selected = [1, 2, 4], total value = 240, total weight = 45/50, and input arrays for capacity values and capacity values. A note mentions 'Levy's flight $x_i^{t+1} = x_i^t + l \cdot \text{Levy}(2)$ while $d = \text{step size scaling factor}$ '.

Cuckoo Search Algorithm

- Meta-heuristic optimization inspired by cuckoo birds laying eggs in other birds' nests to find better solutions.
- It uses Lévy flights to generate new solutions & keeps best nests for next generation.
- Poor solutions are replaced based on discovery probability.

Algorithm:

- 1) Set the initial value of the host nest size n , probability $pa \in (0,1)$ & max no. of iteration $Maxt$.
- 2) Set $t=0$ (counter initialization)
- 3) For $i=1$ to n do
- 4) Generate initial population of n host x_i^t
- 5) Evaluate fitness function $f(x_i^t)$
- 6) End for
- 7) Generate a new solution (cuckoo) x_i^{t+1} randomly by Lévy Flight
- 8) Evaluate fitness function x_i^{t+1} i.e. $f(x_i^{t+1})$
- 9) Choose a nest x_j among n solutions randomly
- 10) If $f(x_i^{t+1}) < f(x_j^t)$ then
- 11) Replace solution x_j with solution x_i^{t+1}
- 12) End if
- 13) Abandon a fraction pa of worst nest
- 14) Build new nest at new location using Lévy flight a fraction pa of worse nest.
- 15) Keep the best solution (nest with quality solution)
- 16) Rank the solution & find current best solution
- 17) Set $t=t+1$
- 18) Until $t \geq Maxt$
- 19) Produce the best solution

Fitness evaluation:

```
if (FitnessOfCuckooEgg < FitnessOfWorstEgg) {
    replace Worst egg with Cuckoo
    t = t + 1; // Next generation
}
if (FitnessOfCuckooEgg < FitnessOfHostEgg) {
    // worst case
    CuckooEgg killed or throw away
    Generate new solution again using Lévy Flight
}
```

Output:

```
items selected = [1, 2, 4]
total value = 240
total weight = 45/50
```

Input is `np.array([10, 20, 30, 40, 50])` & `np.array([100, 100, 100, 100])` for capacity values

Levy's flight $x_i^{t+1} = x_i^t + l \cdot \text{Levy}(2)$ while $d = \text{step size scaling factor}$

Code:

```
import numpy as np
```

```
class CuckooSearchKnapsack:
```

```
    def __init__(self, n_nests=25, pa=0.25, max_iter=100):
        self.n_nests = n_nests
        self.pa = pa
        self.max_iter = max_iter
        self.best_nest = None
        self.best_fitness = float('-inf')
        self.fitness_history = []
```

```

def levy_flight(self, dim):
    from math import gamma, sin, pi
    beta = 1.5
    sigma_u = (gamma(1 + beta) * sin(pi * beta / 2) /
                (gamma((1 + beta) / 2) * beta * 2 ** ((beta - 1) / 2))) ** (1 / beta)
    u = np.random.normal(0, sigma_u, dim)
    v = np.random.normal(0, 1, dim)
    return u / np.abs(v) ** (1 / beta)

def repair_solution(self, solution, weights, capacity):
    solution = (solution > 0.5).astype(int)
    while np.sum(solution * weights) > capacity:
        included = np.where(solution == 1)[0]
        if len(included) == 0:
            break
        remove = np.random.choice(included)
        solution[remove] = 0
    return solution

def fitness(self, solution, values, weights, capacity):
    total_weight = np.sum(solution * weights)
    if total_weight > capacity:
        return 0
    return np.sum(solution * values)

def optimize(self, values, weights, capacity):
    n_items = len(values)
    nests = np.random.rand(self.n_nests, n_items)
    fitness_vals = np.zeros(self.n_nests)

    # Evaluate initial nests
    for i in range(self.n_nests):
        repaired = self.repair_solution(nests[i].copy(), weights, capacity)
        fitness_vals[i] = self.fitness(repaired, values, weights, capacity)

    best_idx = np.argmax(fitness_vals)
    self.best_nest = self.repair_solution(nests[best_idx].copy(), weights, capacity)
    self.best_fitness = fitness_vals[best_idx]

    # Main loop
    for t in range(self.max_iter):
        for i in range(self.n_nests):
            step = self.levy_flight(n_items)
            new_nest = nests[i] + 0.01 * step
            new_nest = np.clip(new_nest, 0, 1)

            new_nest_repaired = self.repair_solution(new_nest.copy(), weights, capacity)
            new_fitness = self.fitness(new_nest_repaired, values, weights, capacity)

            # Random nest to compare
            j = np.random.randint(0, self.n_nests)
            if new_fitness > fitness_vals[j]:
                nests[j] = new_nest
                fitness_vals[j] = new_fitness

            if new_fitness > self.best_fitness:

```

```

        self.best_nest = new_nest_repaired.copy()
        self.best_fitness = new_fitness

    # Abandon worst nests
    worst_idx = np.argsort(fitness_vals)[:int(self.pa * self.n_nests)]
    for idx in worst_idx:
        nests[idx] = np.random.rand(n_items)
        repaired = self.repair_solution(nests[idx].copy(), weights, capacity)
        fitness_vals[idx] = self.fitness(repaired, values, weights, capacity)

    self.fitness_history.append(self.best_fitness)

    return self.best_nest, self.best_fitness

if __name__ == "__main__":
    print("==== Cuckoo Search for 0/1 Knapsack Problem ====\n")

    values = np.array([60, 100, 120, 80, 50, 70, 90, 110])
    weights = np.array([10, 20, 30, 15, 12, 18, 25, 22])
    capacity = 80

    print(f"Number of items: {len(values)}")
    print(f"Knapsack capacity: {capacity}")
    print(f"Items (Value, Weight):")

    for i in range(len(values)):
        print(f"Item {i+1}: ({values[i]}, {weights[i]}kg)")

    cs = CuckooSearchKnapsack(n_nests=30, pa=0.25, max_iter=100)
    best_solution, best_value = cs.optimize(values, weights, capacity)

    selected_items = np.where(best_solution == 1)[0]
    total_weight = np.sum(best_solution * weights)

    print(f"\n==== SOLUTION ===")
    print(f"Selected items: {selected_items + 1}")
    print(f"Total value: ${best_value:.0f}")
    print(f"Total weight: {total_weight:.0f}kg / {capacity}kg")

    print(f"\nItem details:")
    for idx in selected_items:
        print(f"Item {idx+1}: Value=${values[idx]}, Weight={weights[idx]}kg")

    print(f"\nConvergence (every 20 iterations):")
    for i in range(0, len(cs.fitness_history), 20):
        print(f"Iter {i+1:3d}: Best Value = ${cs.fitness_history[i]:.0f}")

```

Output:

Number of items: 8
Knapsack capacity: 80
Items (Value, Weight):
Item 1: (\$60, 10kg)
Item 2: (\$100, 20kg)
Item 3: (\$120, 30kg)
Item 4: (\$80, 15kg)
Item 5: (\$50, 12kg)
Item 6: (\$70, 18kg)
Item 7: (\$90, 25kg)
Item 8: (\$110, 22kg)

=== SOLUTION ===

Selected items: [5 6 7]
Total value: \$400
Total weight: 55kg / 80kg

Item details:

Item 5: Value=\$50, Weight=12kg
Item 6: Value=\$70, Weight=18kg
Item 7: Value=\$90, Weight=25kg

Convergence (every 20 iterations):

Iter 1: Best Value = \$400
Iter 21: Best Value = \$400
Iter 41: Best Value = \$400
Iter 61: Best Value = \$400
Iter 81: Best Value = \$400

Program 6

The Grey Wolf Optimizer (GWO) algorithm is a swarm intelligence algorithm inspired by the social hierarchy and hunting behavior of grey wolves. It mimics the leadership structure of alpha, beta, delta, and omega wolves and their collaborative hunting strategies. The GWO algorithm uses these social hierarchies to model the optimization process, where the alpha wolves guide the search process while beta and delta wolves assist in refining the search direction. This algorithm is effective for continuous optimization problems and has applications in engineering, data analysis, and machine learning.

Algorithm:

```
GREY WOLF OPTIMIZER

Wolf hierarchy
-  $\alpha$  wolf - leader, guides search
-  $\beta$  wolf - 2nd best, assists alpha
-  $\delta$  wolf - 3rd best, subordinate, follows  $\alpha$  &  $\beta$ 
-  $\omega$  wolf - Followers, explore search space

The application I'm using is 'Image processing':
Code:
import numpy as np
def f_obs(x, hist_norm, total_mean, levels):
    thresholds = np.sort(np.clip(np.round(x).astype(int), 0, 255))
    levels[-1] = thresholds

    variance_sum = 0
    for i in range(len(levels)-1):
        start, end = levels[i], levels[i+1]
        w_i = np.sum(hist_norm[start:end])
        mu_i = (np.sum(np.arange(start, end) * hist_norm[start:end]) / w_i) if w_i > 0 else 0
        variance_sum += w_i * (mu_i - total_mean) ** 2
    return variance_sum

hist = np.zeros(256); hist[50:100], hist[150:200] = 500, 1000
hist_norm = hist / np.sum(hist)
total_mean = np.sum(np.arange(256) * hist_norm)
levels = np.zeros(4, dtype=int); levels[-1] = 256
DIM, N_WOLVES, MAX_ITER, LB, UB = 2, 30, 150, 1, 254

wolves = np.random.uniform(LB, UB, (N_WOLVES, DIM))
fitness = np.array([f_obs(w, hist_norm, total_mean, levels) for w in wolves])
alpha, beta, delta = np.argsort(fitness)[-3:]

for l in range(MAX_ITER):
    a = 2 * (1 - l / MAX_ITER)
    for i in range(N_WOLVES):
        X_sum = np.zeros(DIM)
        for j, leader in enumerate([alpha, beta, delta]):
            r1, r2 = np.random.rand(DIM), np.random.rand(DIM)
            A = 2 * a * (r1 - 0.5)
            B = 2 * r2
            D = abs((X_sum + wolves[leader] - A * X_sum))
            X_sum += wolves[leader] - A * D
        wolves[i] = np.clip(X_sum / 3, LB, UB)
        fitness = np.array([f_obs(w, hist_norm, total_mean, levels) for w in wolves])
        alpha, beta, delta = np.argsort(fitness)[-3:]

    best_position = np.sort(np.round(wolves[alpha]).astype(int))
    best_fitness = fitness[alpha]

    print(f"Best position: {best_position}")
    print(f"Best fitness (Max Variance): {best_fitness}")

Output:
Best position: [170 116 175]
Best fitness: 2326.3829
```

Code:

```
import numpy as np

def f_otsu(x, hist_norm, total_mean, levels):
    thresholds = np.sort(np.clip(np.round(x).astype(int), 0, 255))
    levels[1:-1] = thresholds

    variance_sum = 0
    for i in range(len(levels) - 1):
        start, end = levels[i], levels[i+1]
        w_i = np.sum(hist_norm[start:end])
        mu_i = (np.sum(np.arange(start, end) * hist_norm[start:end]) / w_i) if w_i > 0 else 0
        variance_sum += w_i * (mu_i - total_mean)**2

    return -variance_sum

hist = np.zeros(256); hist[50:100], hist[150:200] = 500, 1000
hist_norm = hist / np.sum(hist)
total_mean = np.sum(np.arange(256) * hist_norm)
levels = np.zeros(4, dtype=int); levels[-1] = 256

DIM, N_WOLVES, MAX_ITER, LB, UB = 2, 30, 150, 1, 254

wolves = np.random.uniform(LB, UB, (N_WOLVES, DIM))
fitness = np.array([f_otsu(w, hist_norm, total_mean, levels) for w in wolves])
alpha, beta, delta = np.argsort(fitness)[:3]

for t in range(MAX_ITER):
    a = 2 - 2 * (t / MAX_ITER)
    for i in range(N_WOLVES):
        X_sum = np.zeros(DIM)
        for j, leader in enumerate([alpha, beta, delta]):
            r1, r2 = np.random.rand(DIM), np.random.rand(DIM)
            A = 2 * a * r1 - a
            C = 2 * r2
            D = abs(C * wolves[leader] - wolves[i])
            X_sum += wolves[leader] - A * D

        wolves[i] = np.clip(X_sum / 3, LB, UB)

    fitness = np.array([f_otsu(w, hist_norm, total_mean, levels) for w in wolves])
    alpha, beta, delta = np.argsort(fitness)[:3]

best_position = np.sort(np.round(wolves[alpha]).astype(int))
best_fitness = -fitness[alpha]

print(f"Best position (Optimal Thresholds): {best_position}")
print(f"Best fitness (Max Variance): {best_fitness:.4f}")
```

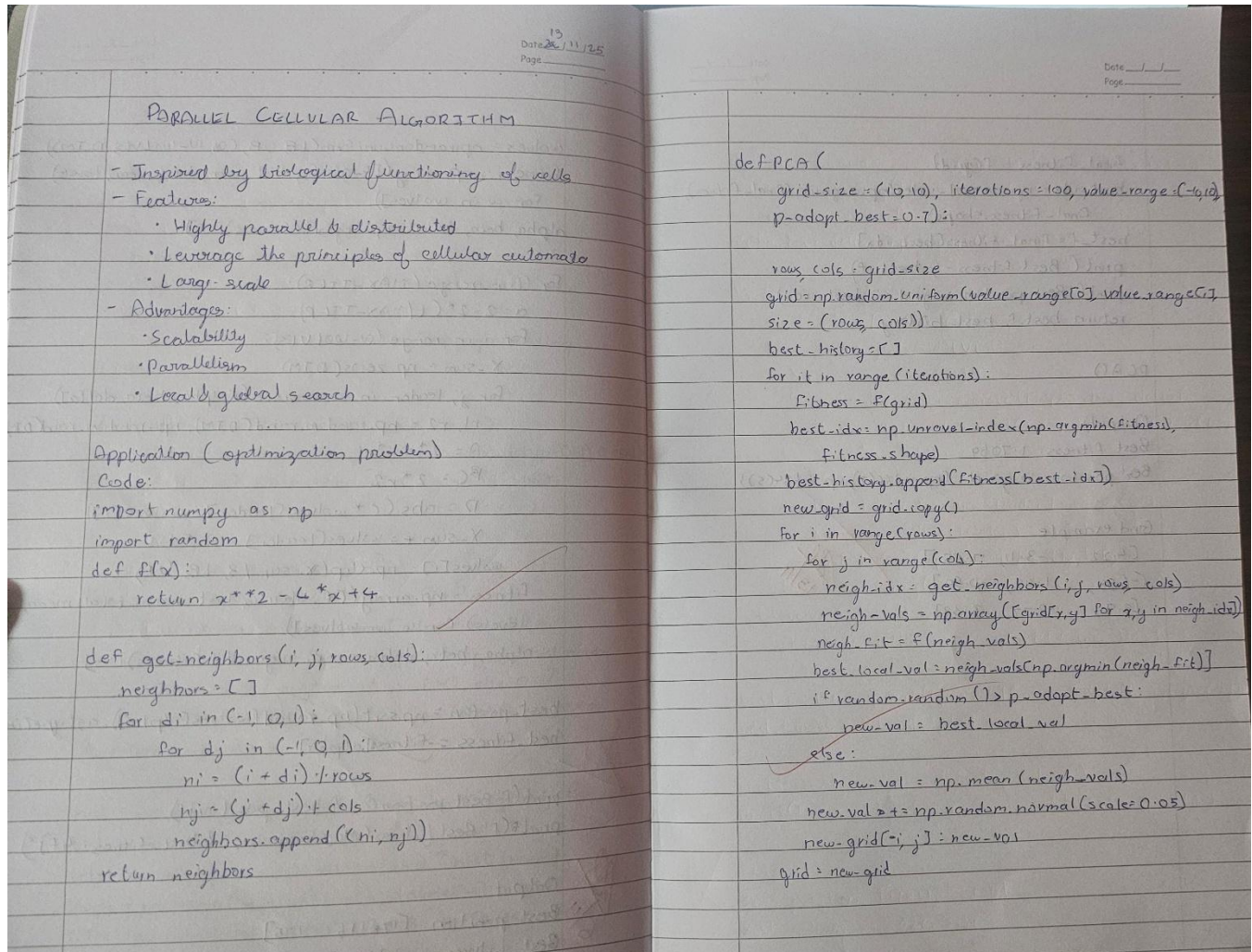
Output:

```
Best position (Optimal Thresholds): [126 175]
Best fitness (Max Variance): 2326.3889
```

Program 7

Parallel Cellular Algorithms are inspired by the functioning of biological cells that operate in a highly parallel and distributed manner. These algorithms leverage the principles of cellular automata and parallel computing to solve complex optimization problems efficiently. Each cell represents a potential solution and interacts with its neighbors to update its state based on predefined rules. This interaction models the diffusion of information across the cellular grid, enabling the algorithm to explore the search space effectively. Parallel Cellular Algorithms are particularly suitable for large-scale optimization problems and can be implemented on parallel computing architectures for enhanced performance.

Algorithm:




```

final_fitness = f(grid)
best_idx = np.unravel_index(np.argmin(final_fitness),
                             final_fitness.shape)
best_f = final_fitness[best_idx]
print("Best fitness =", best_f)
print("Best position in grid =", best_idx)
return best_f, best_history

```

PCA()

Output:

Best fitness = 1.7069

Best position in grid: (np.int64(7), np.int64(5))

Grid example

```

[4.22, -3.11, 7.55],
[-6.98, 1.44, -0.55],
[8.9, -2.77, 3.33]

```

13/11

Code:

```

import numpy as np
import random

```

def f(x):

```

    return x**2 - 4*x + 4

```

def get_neighbors(i, j, rows, cols):

```

    neighbors = []
    for di in (-1, 0, 1):
        for dj in (-1, 0, 1):
            ni = (i + di) % rows
            nj = (j + dj) % cols
            neighbors.append((ni, nj))
    return neighbors

```

def parallel_cellular_algorithm(

```

    grid_size=(10, 10),
    iterations=100,
    value_range=(-10, 10),
    p_adopt_best=0.7

```

):

```

    rows, cols = grid_size

```



```

grid = np.random.uniform(value_range[0], value_range[1], size=(rows, cols))
best_history = []

for it in range(iterations):
    fitness = f(grid)
    best_idx = np.unravel_index(np.argmin(fitness), fitness.shape)
    best_history.append(fitness[best_idx])
    new_grid = grid.copy()

    for i in range(rows):
        for j in range(cols):
            neigh_idx = get_neighbors(i, j, rows, cols)
            neigh_vals = np.array([grid[x, y] for x, y in neigh_idx])
            neigh_fit = f(neigh_vals)
            best_local_val = neigh_vals[np.argmin(neigh_fit)]

            if random.random() < p_adopt_best:
                new_val = best_local_val
            else:
                new_val = np.mean(neigh_vals)

            new_val += np.random.normal(scale=0.05)
            new_grid[i, j] = new_val

    grid = new_grid

final_fitness = f(grid)
best_idx = np.unravel_index(np.argmin(final_fitness), final_fitness.shape)
best_f = final_fitness[best_idx]

print("Best fitness =", best_f)
print("Best position in grid =", best_idx)

return best_f, best_history

parallel_cellular_algorithm()

```

Output:

```

Best fitness = 9.454845972811654e-08
Best position in grid = (np.int64(9), np.int64(9))

```